RBAC & Security

Controlling Access and Hardening Your Cluster

Enterprise EKS Platform — Module 7 of 13

📋 Module Agenda

- Kubernetes security model — authentication, authorization, admission
- RBAC deep dive — Roles, ClusterRoles, Bindings
- ServiceAccounts and pod identity
- Security contexts at pod and container level
- Pod Security Standards and admission controllers
- Security best practices

Kubernetes Security Model

Authentication, authorization, and admission control

🚪 API Request Flow — Three Security Gates

Every request to the API server passes through:

1. Authentication

- Who are you? Validates identity
- Returns user/group info

2. Authorization

- What can you do?
- RBAC evaluates; allow or deny

3. Admission

- Should this be allowed?
- Mutating + validating webhooks

👤 Authentication Methods

X.509 Client Certificates

- CN = username, O = group; used by cluster components

OpenID Connect (OIDC)

- Enterprise SSO via Azure AD, Okta, Dex

Bearer Tokens

- ServiceAccount tokens (JWT) and bootstrap tokens

AWS IAM Authenticator (EKS)

- Maps IAM roles to K8s users via aws-auth ConfigMap

🔒 RBAC Deep Dive

Role-Based Access Control in Kubernetes

📦 RBAC API Objects

Define Permissions

- **Role** — namespace-scoped permissions
- **ClusterRole** — cluster-wide permissions

Grant Permissions

- **RoleBinding** — binds Role/ClusterRole to subjects in a namespace
- **ClusterRoleBinding** — binds ClusterRole to subjects cluster-wide

Key insight: Roles define *what* is allowed. Bindings define *who* gets those permissions. You always need both.

RoleBinding vs ClusterRoleBinding

Subjects can be Users, Groups, or ServiceAccounts. **roleRef** can point to a Role or ClusterRole. ClusterRoleBindings grant permissions across ALL namespaces -- use sparingly.

📈 Verbs and Resources Matrix

Verb	HTTP Method	Description
`get`	GET (single)	Retrieve a specific resource
`list`	GET (collection)	List resources
`watch`	GET (watch)	Stream changes to resources
`create`	POST	Create a new resource
`update`	PUT	Replace entire resource
`patch`	PATCH	Modify part of a resource
`delete`	DELETE	Remove a resource

> **Wildcard:** Use "*" for all verbs or all resources — avoid this outside of testing.

🛠️ Aggregated ClusterRoles

How it works

The controller manager automatically combines rules from all ClusterRoles matching the label selector. Built-in roles like `admin`, `edit`, and `view` use aggregation.

🧰 ServiceAccounts

Identity for workloads running in the cluster

🔭 ServiceAccount Anatomy

Key Properties

- Namespace-scoped identity
- Every namespace has a `default` SA
- Tokens are projected as volumes
- Annotations enable cloud integrations
- Can disable auto-mount for security

🔗 Binding Roles to ServiceAccounts

Pattern: Create a dedicated ServiceAccount per application, grant only the permissions it needs, and reference it in the pod spec with `serviceAccountName`.

☁️ Pod Identity — IRSA on EKS

IAM Roles for Service Accounts

- Maps K8s ServiceAccount to IAM Role
- Uses OIDC federation
- No shared node-level credentials
- Fine-grained per-pod access

How IRSA Works

1. EKS cluster has an OIDC provider
2. IAM role trusts the OIDC provider
3. SA annotated with IAM role ARN
4. Pod receives projected OIDC token
5. AWS SDK exchanges token for creds

Security Context

Constraining what pods and containers can do

 Pod SecurityContext

runAsUser/runAsGroup: Sets the UID/GID for all containers in the pod

fsGroup: Sets the group ownership of all mounted volumes

📦 Container SecurityContext

Container-level overrides pod-level settings

- **allowPrivilegeEscalation:** Prevents gaining more privileges than parent
- **readOnlyRootFilesystem:** Prevents writes to container filesystem
- **capabilities:** Fine-grained Linux capability control
- **privileged:** Never set to `true` for application workloads

📦 Building Secure Container Images

Minimal Images

- **Multi-stage builds** — compile in one stage, copy only the binary to the final image
- **Distroless** — Google's images with no shell or package manager
- **Alpine** — ~5 MB base; good for interpreted languages
- **scratch** — empty base for statically linked Go / Rust binaries

> **Result:** A distroless multi-stage build yields a ~15 MB final image with no shell, no root, no package manager

🔎 Image Security Pipeline

🛡️ Run as Non-Root

- Add USER in Dockerfile
- Pair with `runAsNonRoot: true`
- Never store secrets in layers

🔍 Image Scanning

- **Trivy** — CI-friendly scanner
- **ECR scanning** — on push or continuous
- Scan in CI *and* in registry

✏️ Image Signing

- **Cosign** — keyless OIDC signing
- Verify at admission
- Pin by `@sha256:` digest

Supply-chain rule of thumb: Build minimal → scan for CVEs → sign the artifact → verify at deploy

📐 Pod Security Standards

Privileged

- Unrestricted; all capabilities allowed
- For CNI plugins, storage drivers only

Baseline

- Prevents known escalations
- Good starting point for migration

Restricted

- Non-root, drop all capabilities
- Recommended hardening target

Recommendation: Target **Restricted** for application workloads. Use **Baseline** as a transition step. Reserve **Privileged** for infrastructure components only.

Enforcing: Apply `pod-security.kubernetes.io/` labels to a namespace — `enforce` rejects violations, while `audit` and `warn` only

✅ Security Best Practices

Hardening your cluster

🔐 Principle of Least Privilege in Practice

RBAC Least Privilege

- Use Roles over ClusterRoles when possible
- Grant specific verbs — avoid `*`
- Name specific resources; use `resourceNames` for fine-grained control
- Audit with `kubectl auth can-i` and `--as` to test effective permissions

📝 Audit Logging

Audit Levels

- **None:** Skip logging
- **Metadata:** Request metadata only
- **Request:** Metadata + request body
- **RequestResponse:** Log everything

Guidance: Log RBAC changes at **RequestResponse**; log secrets at **Metadata** only to avoid exposing values.

📡 Image Security

Image Scanning

- Scan in CI/CD pipeline (Trivy, Grype)
- ECR image scanning (on push)
- Block critical CVEs from deploying
- Scan base images regularly

Image Signing & Verification

- Sign images with Cosign / Notation
- Verify signatures at admission
- Use allowlisted registries only
- Pin images by SHA256 digest

> **Always use:**
> `imagePullPolicy: Always`
> with immutable tags or digests

⚡ EKS Security Best Practices

Cluster Configuration

- Enable envelope encryption for secrets
- Use private API endpoint
- Enable control plane logging
- Restrict CIDR for API access
- Keep Kubernetes version current

Workload Configuration

- Use IRSA or EKS Pod Identity
- Enforce Pod Security Standards
- Use NetworkPolicies (needs CNI support)
- Rotate ServiceAccount tokens
- Use external secrets (Secrets Manager)

aws-auth ConfigMap: Regularly audit this to ensure only authorized IAM principals have cluster access. Consider migrating to EKS access entries for more granular control.

🔑 IRSA: IAM Roles for Service Accounts

Least-privilege AWS access for pods

⚠️ Why Node-Level IAM Is Dangerous

> **Without IRSA:** Every pod on a node inherits the EC2 instance profile — the same IAM role for all workloads.

- **Over-privileged pods** — a logging sidecar gets the same S3/SQS access as your data pipeline
- **Blast radius** — one compromised container can access every AWS resource the node role permits
- **No audit granularity** — CloudTrail shows the node role, not which pod made the API call

IRSA solves this by mapping each Kubernetes ServiceAccount to its own IAM role with scoped permissions — true least privilege for AWS access.

🔀 How IRSA Works: OIDC → STS → Credentials

End-to-End Flow

1. **EKS cluster exposes an OIDC endpoint** — registered as an IAM Identity Provider
2. **Pod starts** — EKS webhook injects a projected JWT token + `AWS_ROLE_ARN` env var
3. **AWS SDK calls STS** `AssumeRoleWithWebIdentity` with the JWT
4. **STS validates** the token signature via the OIDC provider's JWKS
5. **STS returns temporary credentials** scoped to that ServiceAccount's IAM role

> **Zero code changes:** The AWS SDK credential chain detects IRSA environment variables automatically

Configuring IRSA

1. Annotate the ServiceAccount

Add the `eks.amazonaws.com/role-arn` annotation. It tells the EKS webhook which IAM role to inject into pods using this SA.

2. IAM Trust Policy

The IAM role trusts the cluster's OIDC provider via `sts:AssumeRoleWithWebIdentity`, with `:aud` and `:sub` conditions scoping it to the exact ServiceAccount.

Security boundary: The `:sub` condition scopes the role to an exact namespace + ServiceAccount. Without it, any SA in the cluster could assume this role.

✅ IRSA Best Practices

Do

- One IAM role per workload
- Scope policies to specific resource ARNs
- Always include `:sub` condition in trust policies
- Verify with `aws sts get-caller-identity`

Avoid

- Sharing the node instance profile
- Wildcard actions or `"Resource": "*"`
- Broad `StringLike` on `:sub`
- Long-lived AWS keys in Secrets

EKS Pod Identity (GA 2023) simplifies IRSA by removing trust-policy management. IRSA remains the standard for existing clusters.

🔧 Lab 7: RBAC and Security

Hands-on: Securing your cluster workloads

- Create Roles and RoleBindings for namespace-scoped access
- Test permissions with `kubectl auth can-i --as`
- Create dedicated ServiceAccounts for applications
- Apply security contexts to pods and containers
- Configure Pod Security Admission on a namespace
- Verify that non-compliant pods are rejected
- Annotate a ServiceAccount for IRSA and verify scoped AWS credentials

> **Duration:** ~45-55 minutes

🎯 Key Takeaways

Least Privilege: Grant minimum permissions needed. Roles define what; Bindings define who. Start with no access and add as required.

PSS Restricted Profile: Use the restricted Pod Security Standard to enforce non-root, read-only FS, and no privilege escalation. Combine with securityContext and audit logging for defense in depth.

IRSA for Pod Identity: Per-pod AWS credentials via OIDC federation eliminate shared node IAM roles and static credentials.

Audit Logging: Enable audit logging and forward to your SIEM for detecting unauthorized access and policy violations.

? Questions & Discussion

How does your team currently manage access control in your Kubernetes clusters?

Up Next: Module 8 — Network Policies